FULL STACK DEVELOPMENT WITH MERN

PROJECT DOCUMENTATION

Online Complaint Registration System

Prepared By: THUM SUCHITH REDDY

Project Documentation for Academic Submission

1. Introduction

The Online Complaint Registration System is a web-based platform developed using the MERN Stack. It provides a centralized mechanism for users to register complaints, track their progress, and receive updates. The system reduces paperwork, increases transparency, and improves complaint resolution efficiency.

Detailed Discussion: The Online Complaint Registration System is a web-based platform developed using the MERN Stack. It provides a centralized mechanism for users to register complaints, track their progress, and receive updates. The system reduces paperwork, increases transparency, and improves complaint resolution efficiency. Detailed Discussion: The Online Complaint Registration System is a web-based platform developed using the MERN Stack. It provides a centralized mechanism for users to register complaints, track their progress, and receive updates. The system reduces paperwork, increases transparency, and improves complaint resolution efficiency. Detailed Discussion: The Online Complaint Registration System is a web-based platform developed using the MERN Stack. It provides a centralized mechanism for users to register complaints, track their progress, and receive updates. The system reduces paperwork, increases transparency, and improves complaint resolution efficiency.

2. Project Overview

The project aims to digitize complaint management. Users can submit complaints online while administrators manage and resolve them through a dashboard. The platform supports authentication, complaint tracking, notifications, and reporting.

Detailed Discussion: The project aims to digitize complaint management. Users can submit complaints online while administrators manage and resolve them through a dashboard. The platform supports authentication, complaint tracking, notifications, and reporting. Detailed Discussion: The project aims to digitize complaint management. Users can submit complaints online while administrators manage and resolve them through a dashboard. The platform supports authentication, complaint tracking, notifications, and reporting. Detailed Discussion: The project aims to digitize complaint management. Users can submit complaints online while administrators manage and resolve them through a dashboard. The platform supports authentication, complaint tracking, notifications, and reporting.

3. Problem Statement

Traditional complaint handling methods rely on paper forms, emails, and manual processes. These approaches lead to delays, poor tracking, and lack of accountability. A centralized digital system is required.

Detailed Discussion: Traditional complaint handling methods rely on paper forms, emails, and manual processes. These approaches lead to delays, poor tracking, and lack of accountability. A centralized digital system is required. Detailed Discussion: Traditional complaint handling methods rely on paper forms, emails, and manual processes. These approaches lead to delays, poor tracking, and lack of accountability. A centralized digital system is required. Detailed Discussion: Traditional complaint handling methods rely on paper forms, emails, and manual processes. These approaches lead to delays, poor tracking, and lack of accountability. A centralized digital system is required.

4. Objectives

• Digitize complaint management
• Improve transparency
• Enable complaint tracking
• Reduce manual effort
• Improve communication
• Increase user satisfaction

Detailed Discussion: • Digitize complaint management
• Improve transparency
• Enable complaint tracking
• Reduce manual effort
• Improve communication
• Increase user satisfaction Detailed Discussion: • Digitize complaint management
• Improve transparency
• Enable complaint tracking
• Reduce manual effort
• Improve communication
• Increase user satisfaction Detailed Discussion: • Digitize complaint management
• Improve transparency
• Enable complaint tracking
• Reduce manual effort
• Improve communication
• Increase user satisfaction

5. System Architecture

The system follows a three-tier architecture:
Presentation Layer (React.js)
Business Logic Layer (Node.js + Express.js)
Database Layer (MongoDB Atlas)

Flow: User → Frontend → Backend APIs → Database → Response

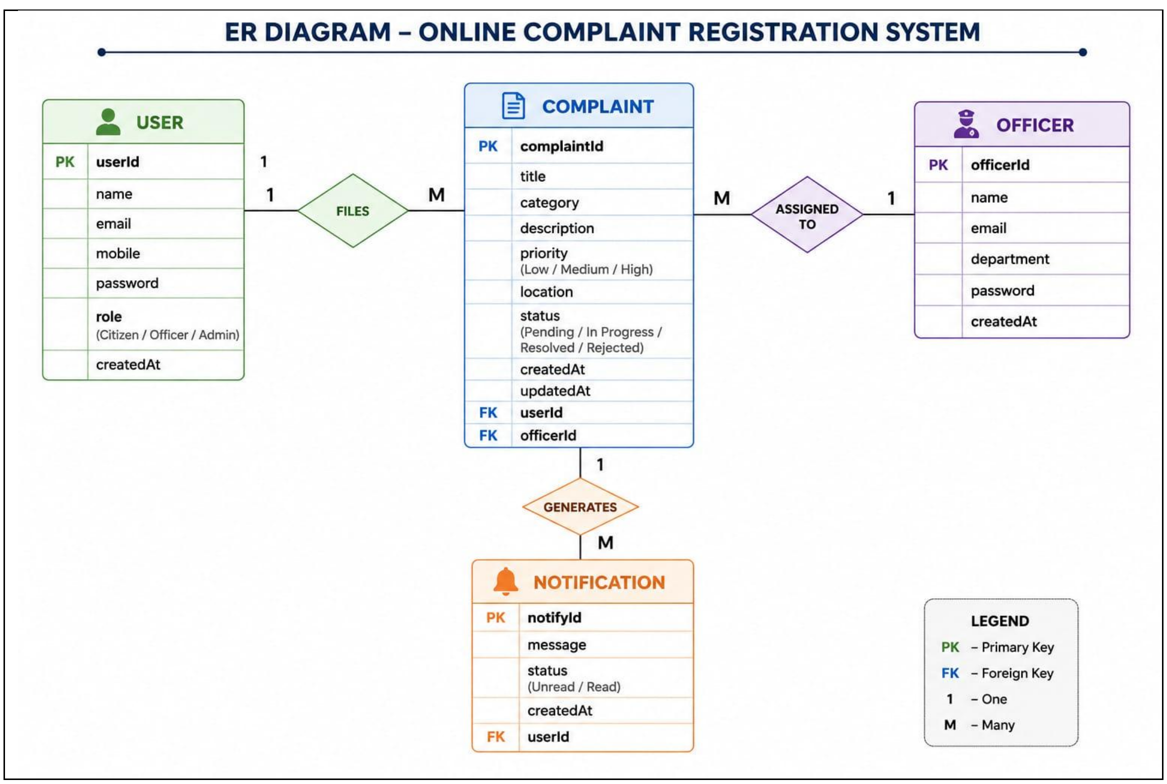
Detailed Discussion: The system follows a three-tier architecture:
Presentation Layer (React.js)
Business Logic Layer (Node.js + Express.js)
Database Layer (MongoDB Atlas)

Flow: User → Frontend → Backend APIs → Database → Response Detailed Discussion: The system follows a three-tier architecture:
Presentation Layer (React.js)
Business Logic Layer (Node.js + Express.js)
Database Layer (MongoDB Atlas)

Flow: User → Frontend → Backend APIs → Database → Response Detailed Discussion: The system follows a three-tier architecture:
Presentation Layer (React.js)
Business Logic Layer (Node.js + Express.js)
Database Layer (MongoDB Atlas)

Flow: User → Frontend → Backend APIs → Database → Response

Er diaram



6. Technology Stack

Frontend: React.js, Vite, Axios, React Router DOM
Backend: Node.js, Express.js
Database: MongoDB Atlas, Mongoose
Authentication: JWT, bcryptjs
Deployment: Vercel, Render
Version Control: Git, GitHub

Detailed Discussion: Frontend: React.js, Vite, Axios, React Router DOM
Backend: Node.js, Express.js
Database: MongoDB Atlas, Mongoose
Authentication: JWT, bcryptjs
Deployment: Vercel, Render
Version Control: Git, GitHub Detailed Discussion: Frontend: React.js, Vite, Axios, React Router DOM
Backend: Node.js, Express.js
Database: MongoDB Atlas, Mongoose
Authentication: JWT, bcryptjs
Deployment: Vercel, Render
Version Control: Git, GitHub Detailed Discussion: Frontend: React.js, Vite, Axios, React Router DOM
Backend: Node.js, Express.js
Database: MongoDB Atlas, Mongoose
Authentication: JWT, bcryptjs
Deployment: Vercel, Render
Version Control: Git, GitHub

7. Setup Instructions

Install Node.js, Git, and MongoDB Atlas account.
Clone repository.
Install dependencies using npm install.
Configure .env variables.
Run frontend and backend servers.

Detailed Discussion: Install Node.js, Git, and MongoDB Atlas account.
Clone repository.
Install dependencies using npm install.
Configure .env variables.
Run frontend and backend servers. Detailed Discussion: Install Node.js, Git, and MongoDB Atlas account.
Clone repository.
Install dependencies using npm install.

Configure .env variables.
Run frontend and backend servers. Detailed Discussion: Install Node.js, Git, and MongoDB Atlas account.
Clone repository.
Install dependencies using npm install.
Configure .env variables.
Run frontend and backend servers.

8. Folder Structure

Client: components, pages, services, routes
Server: config, controllers, middleware, models, routes
This structure ensures maintainability and scalability.

Detailed Discussion: Client: components, pages, services, routes
Server: config, controllers, middleware, models, routes
This structure ensures maintainability and scalability. Detailed Discussion: Client: components, pages, services, routes
Server: config, controllers, middleware, models, routes
This structure ensures maintainability and scalability. Detailed Discussion: Client: components, pages, services, routes
Server: config, controllers, middleware, models, routes
This structure ensures maintainability and scalability.

9. Application Workflow

User Registration → Login → Submit Complaint → Complaint Review → Status Update → Resolution → Notification

Detailed Discussion: User Registration → Login → Submit Complaint → Complaint Review → Status Update → Resolution → Notification Detailed Discussion: User Registration → Login → Submit Complaint → Complaint Review → Status Update → Resolution → Notification Detailed Discussion: User Registration → Login → Submit Complaint → Complaint Review → Status Update → Resolution → Notification

10. API Documentation

Authentication APIs:
POST /register
POST /login

Complaint APIs:
POST /complaints
GET /complaints
PUT /complaints/:id

Admin APIs:

GET /all-complaints
PUT /status-update

Detailed Discussion: Authentication APIs:
POST /register
POST /login

Complaint APIs:
POST /complaints
GET /complaints
PUT /complaints/:id

Admin APIs:
GET /all-complaints
PUT /status-update Detailed Discussion: Authentication APIs:
POST /register
POST /login

Complaint APIs:
POST /complaints
GET /complaints
PUT /complaints/:id

Admin APIs:
GET /all-complaints
PUT /status-update Detailed Discussion: Authentication APIs:
POST /register
POST /login

Complaint APIs:
POST /complaints
GET /complaints
PUT /complaints/:id

Admin APIs:
GET /all-complaints
PUT /status-update

11. Authentication & Security

JWT Authentication, bcrypt password hashing, protected routes, role-based access control, secure API communication.

Detailed Discussion: JWT Authentication, bcrypt password hashing, protected routes, role-based access control, secure API communication. Detailed Discussion: JWT Authentication, bcrypt password hashing, protected routes, role-based access control, secure API communication. Detailed Discussion: JWT Authentication, bcrypt password hashing, protected routes, role-based access control, secure API communication.

12. User Interface

The application includes Login Page, Registration Page, User Dashboard, Complaint Submission Form, Complaint Tracking Page, Admin Dashboard, and Notification Center.

Detailed Discussion: The application includes Login Page, Registration Page, User Dashboard, Complaint Submission Form, Complaint Tracking Page, Admin Dashboard, and Notification Center. Detailed Discussion: The application includes Login Page, Registration Page, User Dashboard, Complaint Submission Form, Complaint Tracking Page, Admin Dashboard, and Notification Center. Detailed Discussion: The application includes Login Page, Registration Page, User Dashboard, Complaint Submission Form, Complaint Tracking Page, Admin Dashboard, and Notification Center.

13. Testing

Functional Testing, API Testing, Integration Testing, and User Acceptance Testing were performed. All major modules passed successfully.

Detailed Discussion: Functional Testing, API Testing, Integration Testing, and User Acceptance Testing were performed. All major modules passed successfully. Detailed Discussion: Functional Testing, API Testing, Integration Testing, and User Acceptance Testing were performed. All major modules passed successfully. Detailed Discussion: Functional Testing, API Testing, Integration Testing, and User Acceptance Testing were performed. All major modules passed successfully.

14. Deployment

Frontend: https://vip-c2-online-complaint-registerati.vercel.app/
Backend: https://complaint-registeration.onrender.com
Database: MongoDB Atlas

Detailed Discussion: Frontend: https://vip-c2-online-complaint-registerati.vercel.app/
Backend: https://complaint-registeration.onrender.com
Database: MongoDB Atlas Detailed Discussion: Frontend: https://vip-c2-online-complaint-registerati.vercel.app/
Backend: https://complaint-registeration.onrender.com
Database: MongoDB Atlas Detailed Discussion: Frontend: https://vip-c2-online-complaint-registerati.vercel.app/
Backend: https://complaint-registeration.onrender.com
Database: MongoDB Atlas

15. Known Issues

Email notifications and SMS integration are not yet implemented. Mobile optimization can be improved further.

Detailed Discussion: Email notifications and SMS integration are not yet implemented. Mobile optimization can be improved further. Detailed Discussion: Email notifications and SMS integration are not yet implemented. Mobile optimization can be improved further. Detailed Discussion: Email notifications and SMS integration are not yet implemented. Mobile optimization can be improved further.

16. Future Enhancements

AI-based complaint categorization, analytics dashboard, SMS alerts, mobile application, multilingual support, and automated complaint routing.

Detailed Discussion: AI-based complaint categorization, analytics dashboard, SMS alerts, mobile application, multilingual support, and automated complaint routing. Detailed Discussion: AI-based complaint categorization, analytics dashboard, SMS alerts, mobile application, multilingual support, and automated complaint routing. Detailed Discussion: AI-based complaint categorization, analytics dashboard, SMS alerts, mobile application, multilingual support, and automated complaint routing.

17. Conclusion

The Online Complaint Registration System successfully demonstrates the implementation of a full-stack MERN application. It improves transparency, efficiency, and user satisfaction while providing a scalable architecture for future growth.

Detailed Discussion: The Online Complaint Registration System successfully demonstrates the implementation of a full-stack MERN application. It improves transparency, efficiency, and user satisfaction while providing a scalable architecture for future growth. Detailed Discussion: The Online Complaint Registration System successfully demonstrates the implementation of a full-stack MERN application. It improves transparency, efficiency, and user satisfaction while providing a scalable architecture for future growth. Detailed Discussion: The Online Complaint Registration System successfully demonstrates the implementation of a full-stack MERN application. It improves transparency, efficiency, and user satisfaction while providing a scalable architecture for future growth.

Live Project Links

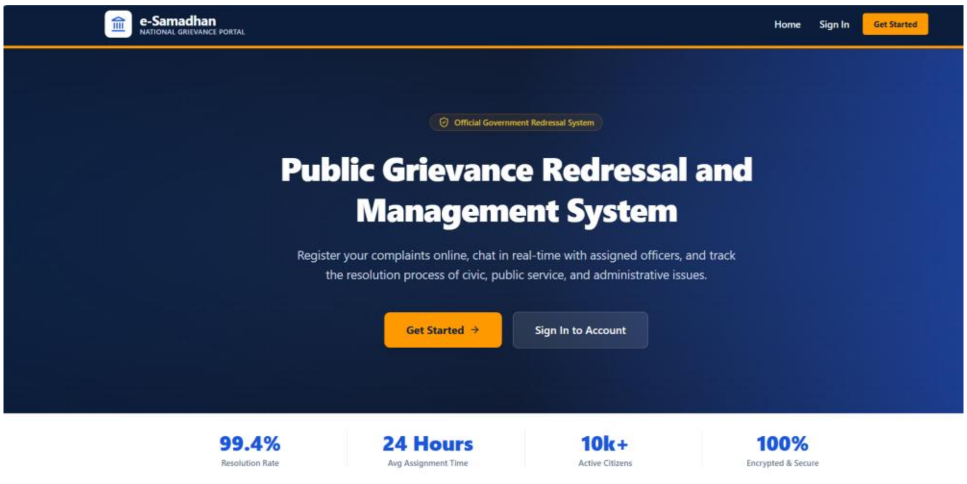
Frontend: https://vip-c2-online-complaint-registerati.vercel.app/

Backend: https://complaint-registeration.onrender.com

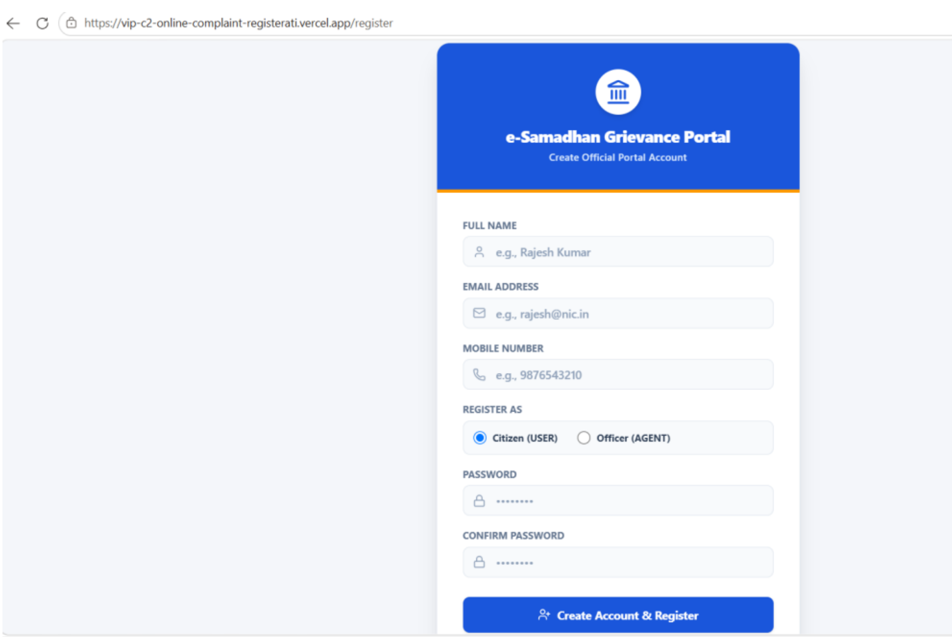
GitHub: https://github.com/suchithreddy-07/-VIP-C2--Online-complaint-registration

ONLINE COMPLAINT REGISTRATION SYSTEM

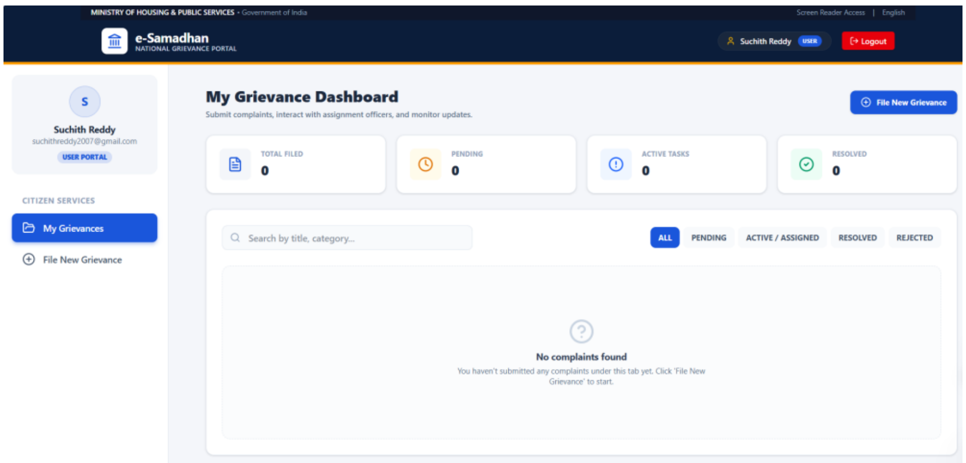
Home Page



Registration Page



User Dashboard



File New Grievance